TP 20 : IF-expressions

Ce TP est à réaliser en OCaml.

Manipuler des IF-expressions

On considère des formules de logique propositionnelle sur n variables notées X_i avec $0 \leq i < n$. Une valeur de vérité est un élément de $\mathbb{B} = \{\top, \bot\}$. Une valuation est une fonction v de $\{0, 1, \ldots, n-1\}$ dans $\mathbb{B}$, qui assigne une valeur de vérité à chacune des variables. On note $V_v(f)$ la valeur de vérité de la formule f pour la valuation v.

On représente les formules propositionnelles en OCaml à l'aide du type suivant :

```
type formule = Const of bool
               | Var of int
               | Neg of formule
               | Et of formule * formule
               | Ou of formule * formule
               | Imp of formule * formule

type valuation = bool array
```

Le type `valuation` correspond à une valuation. Une valeur de type `valuation` est un tableau de taille n où la i ème case du tableau donne la valeur de la variable X_i.

On cherche à étudier si des formules sont des tautologies.

Question 1 : Écrire une fonction `satisfaite : formule -> bool array -> bool` qui, étant donnée une formule et une valuation de ses variables, teste si celle-ci la satisfait.

Pour cela, on se propose d'introduire la notion de IF-expression. Une IF-expression est définie inductivement de la façon suivante :

- Une constante booléenne est une IF-expression.
- Une Variable est une IF-expression.
- Si m, p et q sont des IF-expressions, if m then p else q en est une aussi.

On représente les IF-expressions par le type suivant :

```
type ifexpression = Cst of bool
                  | Vr of int
                  | If of ifexpression * ifexpression * ifexpression
```

Comme on s'en doute, la valeur d'une IF-expression est la valeur de p si m est vrai et la valeur de q sinon.

FIGURE 1 – Pas d'image cette fois-ci, je n'avais pas d'idées.

Toute formule est équivalente à une IF-expression. En effet, on dispose des équivalences suivantes :

$$\neg p \iff \text{if } p \text{ then } \bot \text{ else } \top$$

$$p \wedge q \iff \text{if } p \text{ then } q \text{ else } \bot$$

$$p \vee q \iff \text{if } p \text{ then } \top \text{ else } q$$

$$p \Rightarrow q \iff \text{if } p \text{ then } q \text{ else } \top$$

Question 2 : Écrire une fonction `formule_to_if : formule -> ifexpression` qui, étant donné une formule propositionnelle, renvoie une IF-expression équivalente.

Question 3 : Proposer des IF-expressions équivalentes aux formules suivantes :

- $p_1 = (X_0 \wedge X_1) \Rightarrow X_0$

- $p_2 = X_0 \Rightarrow (X_0 \wedge X_1)$
- $p_3 = (X_0 \Rightarrow (X_0 \Rightarrow X_1)) \Rightarrow X_1$

On dit qu'une IF-expression est normale si elle est soit atomique(une variable ou une constante), soit de la forme if s then p else q où s est une variable et p et q deux IF-expressions normales.

Question 4 : Écrire une fonction `est_normale : ifexpression -> bool` qui, étant donnée une ifexpression, teste si elle est sous forme normale.

Question 5 : Écrire une fonction `normaliser : ifexpression -> ifexpression` qui, étant donnée une IF-expression renvoie une IF-expression sous forme normale équivalente.

On suppose par la suite que toutes les IF-expressions manipulées sont sous forme normale.

IF-expressions ordonnées

On souhaite maintenant ajouter la contrainte que toutes les IF-expressions que l'on manipule sont ordonnées, au sens où si elle est de la forme if x then p else q, alors les formules p et q ne font intervenir que des variables strictement plus grandes que i et sont aussi ordonnées.

Ainsi, if 0 then (if 1 then $\perp$ else $\top$) else (if 2 then $\top$ else $\perp$) est une formule ordonnée tandis que if 0 then (if 1 then $\perp$ else $\top$) else (if 0 then $\top$ else $\perp$) n'en est pas une.

Question 6 : Écrire une fonction `est_ordonnee : ifexpression -> int -> bool` qui, étant donnés une IF-expression et un entier n, qui teste si f est bien ordonnée et n'utilise que des variables d'indices supérieurs à n.

Afin de déterminer si une formule est une tautologie, on se donne le type suivant :

```
type result = Tautologie | Refutation of valuation
```

Question 7 : Écrire une fonction `decide : int -> ifexpression -> result` qui, étant donné un entier n et une IF-expression ordonnée à n variables, renvoie :

- `Tautologie` si f est une tautologie.
- `Refutation v` sinon, avec $V_v(f) = \perp$.

On cherchera à exploiter le caractère ordonnée de la formule pour éviter de considérer le test systématique de toutes les valuations.

On cherche maintenant à construire une fonction, qui, étant donnée une formule propositionnelle, renvoie une IF-expression ordonnée équivalente.

Question 8 : Écrire une fonction `ordonne_not : ifexpression -> ifexpression` qui, étant donnée une IF-expression ordonnée, renvoie une IF-expression ordonnée équivalente à sa négation.

Question 9 : Écrire une fonction `ordonne_or : ifexpression -> ifexpression -> ifexpression` qui, étant donnés deux IF-expressions ordonnées, renvoie une formule ordonnée équivalente à leur disjonction.

Question 10 : Écrire une fonction `ordonne : formule -> ifexpression` qui, étant donnée une formule propositionnelle, renvoie une IF-expression ordonnée équivalente.

Petite application

On propose pour mettre en application les algorithmes développés jusqu'à présent. Pour cela, on utilise l'exemple du sujet 0 du concours X-ENS.

Le poisson d'avril est une espèce extrêmement rare qui n'a jamais été permis d'observer dans la nature. Néanmoins, les ichtyologues ont permis de déterminer les faits suivants :

- tout poisson d'avril qui ne nage pas en mer chaude a des rayures rouges ;
- tout poisson d'avril a des nageoires bleues ou n'a pas de rayures rouges ;
- les poissons d'avril qui vivent dans le corail ne mangent pas de crevettes ;
- un poisson d'avril mange des crevettes si et seulement s'il nage en mer chaude ;
- tout poisson d'avril qui a des nageoires bleues nage en mer chaude et vit dans le corail ;
- tout poisson d'avril qui nage en mer chaude a des nageoires bleues.

On souhaite démontrer qu'il n'existe pas de poisson d'avril.

Question 11 : Démontrer ce résultat à l'aide de votre programme (en espérant ne pas avoir fait d'erreurs).